

Construire une application intégrant l'IA

Call To Adventure · Bloc 3 · E4 : 20 minutes

PRODUIT

Une aventure interactive dans le navigateur, avec narration IA et règles de jeu côté serveur.

RÉALISATION

Cadrer, concevoir, organiser et développer les parcours du joueur.

QUALITÉ / LIVRAISON

Tester le métier et les accès.
Automatiser les contrôles.
Préparer un artefact déployable.

Le bloc 3 relie les besoins du joueur aux preuves de fonctionnement.

Permettre de jouer et de retrouver sa progression

Les parcours distinguent le visiteur et le joueur connecté.

VISITEUR

- Choisir une aventure.
- Créer un personnage valide.
- Jouer sans créer de compte.
- Progression liée à la session.

JOUEUR CONNECTÉ

- Retrouver ses sauvegardes.
- Reprendre une partie active.
- Réutiliser ses personnages.
- Gérer ses propres données.

EXPLOITANT / JURY

- Installer et vérifier le service.
- Comprendre les choix techniques.
- Relier exigences, code, tests et démonstration.

La continuité de la partie et la propriété des données orientent la conception.

Transformer le besoin en critères vérifiables

Exemples issus des exigences et des tests du projet.

REPRENDRE SA PARTIE

Un joueur peut charger sa sauvegarde active.
Un autre utilisateur ne peut ni la charger ni la supprimer.

SURVIVRE À UNE PANNE

Un tour IA en échec retourne une erreur contrôlée.
La progression sauvegardée ne doit pas avancer.

UTILISER L'INTERFACE

Navigation et états compréhensibles.
Critères clavier, focus et annonces.
Cible d'accessibilité à valider.

Chaque exigence doit avoir un composant, un test et une preuve de démonstration.

Séparer présentation, métier et persistance

Une application Django avec des services Python internes.

PRÉSENTATION

Templates HTML / CSS / JS



Vues Django et endpoints JSON



Sessions, auth et CSRF

MÉTIER

GameEngine



LangGraph Combat

IA + RAG règles serveur

STOCKAGE / SERVICES

Base Django : comptes / parties.

SQLite catalogue : références.

Chroma / Ollama : contexte.

Fournisseur externe : génération.

Le navigateur envoie des actions ; le serveur décide des mutations autorisées.

Choisir une architecture adaptée au périmètre

Les choix actuels favorisent une démonstration locale reproductible.

DJANGO

Comptes, sessions, ORM et templates dans un même cadre.
Moins de couches à exploiter pour ce projet solo.

SQLITE / SERVICES IA

Données locales de taille modeste.
IA hébergée et retrieval local.
Limites de concurrence et de dépendance externe documentées.

CIBLE DE DÉPLOIEMENT

Conteneur web et proxy HTTPS.
Secrets externes, stockage durable.
Hébergement et SGBD de production à confirmer.

La préproduction cible est décrite ; son accessibilité réelle reste à prouver.

Piloter un projet solo avec des critères explicites

Backlog, priorités, risques et décisions sont documentés.

PRÉPARER

Besoin formulé.
Critères testables.
Composants concernés.
Risques et preuve attendue.
Définition de prêt.

RÉALISER

Sécurité → cadrage → qualité
→ industrialisation → exploitation.
Suivi par éléments de backlog et
commits.

TERMINER / REVOIR

Code et documentation cohérents.
Tests pertinents.
Preuve conservée.
Notes de revue et rétrospective à
maintenir.

Les rituels, dates et échanges réels doivent compléter les supports de pilotage.

Conserver une progression cohérente

Le moteur expose des transitions de jeu et des règles de persistance.

PARCOURS

Accueil → personnage → récit
↓
Choix → combat éventuel
↓
Victoire, défaite ou poursuite

ÉTAT

Anonyme : session Django.
Connecté : SaveGame + session.
État sérialisé en JSON.
Partie terminée non rechargeable.

RÈGLES MÉTIER

Combat par session.
Soins et dégâts côté serveur.
Propriété des sauvegardes.
Échec IA sans progression persistée.

Les tests portent sur les transitions et les droits, au-delà du rendu des pages.

Protéger les requêtes et les données du joueur

Des contrôles concrets, avec des limites de production identifiées.

REQUÊTES

CSRF sur les POST navigateur.
Taille JSON limitée.
Quota sur les routes IA.
Contenu dynamique via
textContent.

PROPRIÉTÉ / SECRETS

Filtres par utilisateur.
État de combat isolé.
Secret et hôtes de production via
environnement.

LIMITES

Audit dépendances à compléter.
Cache partagé pour plusieurs
instances.
Revue des appels longs et de la
concurrence du graphe.

Les protections du navigateur ont évolué : le code courant prime sur les anciennes notes.

Rendre les interactions perceptibles et maîtrisées

Améliorations présentes ; validation manuelle encore nécessaire.

INTERACTIONS

Annonces de statut dynamiques.
Onglets avec sémantique ARIA.
Focus après mise à jour.
Barres de vie exposées comme progression.

VÉRIFICATION MANUELLE

Parcours au clavier.
Contrastes et zoom 200 %.
Lecteur d'écran.
Qualité des textes FR / EN.

SOBRIÉTÉ

Appels de jeu déclenchés par action.
Réutilisation du retrieval.
Images locales.
Poids et consommation à mesurer.

La présence d'attributs ARIA ne suffit pas à valider l'accessibilité réelle.

44 tests ciblés réussis sur le bloc applicatif

Vérifications locales du 14 septembre 2026.

LOT EXÉCUTÉ

Combat et isolation.
Sécurité des requêtes.
Templates et parcours simulé.
Sauvegardes, personnages,
métriques.

MIGRATIONS

`makemigrations --check`
`--dry-run`

Aucun changement détecté.
Aucune migration créée.

CI VERSIONNÉE

PR, push main / master, manuel.
Lint, checks, tests et couverture.
Dépendances verrouillées.
Résultats distants à joindre.

44 tests ne désignent ni la suite complète ni un pourcentage de couverture.

Construire, vérifier et conserver le même artefact

Le conteneur constitue l'unité de livraison prévue.

BUILD / SMOKE

Dockerfile Python 3.12.
Installation verrouillée.
Démarrage Gunicorn.
Image taguée SHA, test /health
puis export tar en CI.

EXPLOITATION

Configuration par environnement.
Données persistantes à prévoir.
Migrations au démarrage local.
Étape de release à contrôler en
production.

PROMOTION / RETOUR

Déployer le digest vérifié.
Conserver l'image précédente.
Contrôler la compatibilité DB.
Rejouer santé et parcours.

Le job staging est un placeholder : il ne déploie pas encore l'application.

Montrer le parcours, puis les garanties

Séquence préparée : 1 min 40, dans un environnement de démonstration.

00:00 → 00:35

Ouvrir une aventure et un personnage déjà prêts.
Afficher récit et choix.
Exécuter une action de jeu.

00:35 → 01:10

Montrer une sauvegarde du compte de test et sa reprise.
Présenter le résultat du test de propriété.

01:10 → 01:40

Afficher les résultats locaux et le workflow.
Nommer les preuves manquantes.
Introduire le cas d'incident E5.

Prévoir une capture de secours ; annoncer toute simulation utilisée.